

Software Composition Analysis (SCA)

CompTIA SecAI+: AI Security Certification Complete Exam Prep 2026 | Section 21: CI/CD and AI Automation

1. The Dependency Iceberg — Your Real Attack Surface

Most developers think of their application as the code they wrote. In reality, for a modern application, custom code is often a small minority of what executes in production. A Node.js web application may pull in 800 or more npm packages. A Python data service may import 150+ PyPI dependencies. A Java microservice may have hundreds of Maven transitive dependencies. Below each direct dependency lies its own dependency tree — a complex, largely unmapped web of third-party code that your application implicitly trusts.

This is the dependency iceberg: the visible tip is your direct imports; the mass below the waterline is the transitive dependency graph. Software Composition Analysis (SCA) is the sonar system that maps the full iceberg. Without it, you are trusting code written by maintainers you have never met, reviewed by processes you cannot audit, with security practices ranging from expert to none — and you have no visibility into the risk that web represents.

Memory Anchor: The dependency iceberg analogy captures SCA's core value: 90% of your application's code is below the waterline of direct visibility. SCA makes the invisible mass visible and manageable.

2. The XZ Utils Backdoor — A 2024 Watershed Event

Real-World Incident — XZ Utils Backdoor (2024): In March 2024, Andres Freund, a Microsoft engineer, discovered that XZ Utils — a compression library present in virtually every Linux distribution — had been backdoored. The attacker had operated under a false identity for approximately two years, contributing legitimate patches to build reputation and trust within the project community, before inserting a sophisticated backdoor targeting systemd-linked SSH daemons. If successfully deployed at scale, the backdoor would have allowed the attacker to authenticate as any user on affected systems. The attack was discovered by chance: Freund noticed anomalous SSH connection latency during unrelated performance profiling. The XZ Utils package was a transitive dependency in hundreds of millions of systems, illustrating that even unmaintained or slow-moving packages in the deep transitive graph represent a real supply chain attack surface.

The XZ incident demonstrates three AI-relevant SCA capabilities: monitoring for unusual maintainer activity patterns (a new contributor rapidly gaining commit access should be an anomaly signal), detecting unexpected changes in build scripts of established packages (the backdoor was in the build system, not the source), and flagging anomalous package behavior. These signals are exactly what AI-powered SCA monitoring targets.

3. What SCA Tools Do — Core Capabilities

A mature SCA tool delivers a layered set of capabilities. Understanding each layer separately is important for both the exam and practical implementation:

SCA Capability	What It Does	AI Enhancement
Dependency inventory	Parses package manifests and lockfiles to enumerate all direct and transitive dependencies across all ecosystems	AI identifies implicit dependencies not captured in manifests
Vulnerability correlation	Matches dependency inventory against NVD, GitHub Security Advisories, OSV, and ecosystem-specific sources	ML ranking of finding severity in context
Reachability analysis	Determines whether vulnerable functions in dependencies are actually called by your application code	AI call-graph analysis reduces noise by 70-90%
License compliance	Identifies license type for every dependency and flags incompatibilities with your license policy	AI handles complex license expression parsing
Dependency health signals	Assesses maintenance activity, contributor diversity, release cadence, and community health	AI predicts future risk from current health signals
SBOM generation	Produces machine-readable software bill of materials in SPDX or CycloneDX format	AI enriches SBOM with provenance and risk metadata

4. Reachability Analysis — The SCA Game Changer

Traditional SCA tools report every vulnerability in every dependency regardless of whether your application actually exercises the vulnerable code path. A critical CVE in a library function that your code never calls is reported with the same urgency as a critical CVE in a function called directly from your authentication handler. This generates enormous noise — applications with hundreds of dependencies routinely surface thousands of findings, most irrelevant to real risk.

AI-powered reachability analysis changes this through call-graph analysis: the tool traces the actual execution paths from your code into the dependency, determining whether the vulnerable function can be reached. Tools including Snyk, Endor Labs, and Socket perform this analysis. The impact is significant: reachability analysis typically reduces the actionable finding set by 70 to 90 percent in real applications. A developer looking at 20 genuinely exploitable findings behaves very differently from a developer looking at 2,000 theoretical vulnerabilities in imported packages they may never invoke.

5. Transitive Dependencies — The Invisible Majority

Direct dependencies — packages explicitly added to your project's manifest — are at least visible in your package.json, requirements.txt, or pom.xml. Transitive dependencies — the packages your dependencies

import, and the packages those packages import — are largely invisible unless you use dedicated tooling. Yet they represent the majority of your actual dependency surface area.

The XZ Utils backdoor was a transitive dependency for most affected systems: very few projects had directly imported XZ Utils, but it appeared as a transitive dependency of libraries that almost everyone used. AI-powered SCA tools generate complete dependency graphs showing the full transitive tree with depth visualization, identifying exactly which direct dependency introduced each transitive package. This visibility enables informed architectural decisions: a team can determine that replacing one direct dependency would eliminate an entire subtree of problematic transitive dependencies.

6. Continuous Monitoring vs. Point-in-Time Scanning

One of the most tested SCA concepts in the CompTIA SecAI+ exam is the distinction between point-in-time scanning and continuous monitoring. This distinction matters operationally and is a direct exam question topic:

Approach	Description	Key Limitation	Typical Tooling
Point-in-time SCA scan	Scan dependencies at a specific moment (e.g., during CI build)	Misses CVEs disclosed after the scan runs	CLI scanners, manual audits
Continuous SCA monitoring	Tracks your dependency inventory against live vulnerability feeds 24/7	Requires always-on tooling and alert management	GitHub Dependabot, Snyk, Mend
Policy-gated CI/CD enforcement	Blocks PRs and deployments violating defined dependency policy	Requires policy definition and alert tuning	Snyk, Dependabot, OWASP Dependency-Check

A dependency that was clean when your last scan ran may have a critical CVE disclosed the following morning. For regulated industries, continuous monitoring is increasingly a compliance requirement. GitHub Dependabot, Snyk, and Mend all offer continuous monitoring with immediate alerting when new vulnerabilities affect packages in your inventory.

7. License Compliance — The Legal Dimension of SCA

License compliance is frequently underweighted by security teams but represents a distinct category of SCA risk. Open-source licenses carry legal obligations that vary dramatically in their implications for commercial software products:

- Permissive licenses (MIT, Apache 2.0, BSD): Minimal constraints on commercial use. Attribution requirements (including the license text in distributions) are typically the primary obligation.
- Copyleft licenses (GPL, LGPL, AGPL): Derivative works must be released under compatible licenses. Using GPL-licensed code in a proprietary commercial product without fulfilling this obligation creates significant legal exposure.

- Server-Side Public License (SSPL): Used by MongoDB and Elasticsearch. Particularly restrictive terms for organizations offering the software as a cloud service — using SSPL software in a hosted service offering may require open-sourcing the entire service stack.
- Creative Commons licenses: Varied — CC BY allows commercial use with attribution; CC NC prohibits commercial use entirely.

AI-powered SCA tools scan all dependencies for license metadata, flag incompatibilities with your declared license policy, track obligations you must fulfill (attribution requirements, notice files, source code release requirements), and alert when new dependencies introduce incompatible terms. License non-compliance discovered post-distribution can result in injunctions, legal settlements, and mandatory open-sourcing of proprietary code.

8. Automated Remediation — Dependabot, Renovate, and Snyk

The remediation workflow for dependency vulnerabilities is where most SCA programs break down. Development teams discover vulnerabilities through scanning but lack bandwidth to research and implement fixes, creating a growing backlog of known-but-unpatched dependencies. GitHub Dependabot, Renovate Bot, and Snyk's automated PR capability address this directly.

When a new CVE is disclosed affecting a package in your inventory, the tool automatically generates a pull request that updates the package to the patched version, runs your automated test suite against the update, and reports whether the upgrade breaks any existing tests. The developer's task becomes reviewing a proposed upgrade rather than researching a vulnerability class and determining the correct fix. Mean time to remediate is reduced significantly, and the backlog does not accumulate because fixes are proposed automatically at CVE disclosure time rather than waiting for developer bandwidth.

9. Policy Gates and Enforcement Automation

SCA becomes a systemic security control — rather than an advisory report — when integrated as enforcement gates into the CI/CD pipeline. Effective policy gate implementation typically includes:

- Block PRs introducing new dependencies with high or critical CVEs: Developers cannot merge code that adds a vulnerable package without explicit security team override.
- Block introduction of incompatible licenses: Legal policy automatically enforced at the package import point, not discovered post-distribution.
- Require security review for major version upgrades: Automated flagging of significant version changes that may introduce breaking changes or new dependencies.
- Block production deployment with unpatched critical vulnerabilities: The deployment pipeline checks current dependency vulnerability status before allowing production promotion.

These gates make security governance practical at the pace of modern development. A team pushing dozens of pull requests per day cannot perform manual security review for every package addition; automation enforces consistent policy across every PR and every deployment without requiring proportional security team bandwidth.

10. SBOM — The Software Bill of Materials

The Software Bill of Materials (SBOM) has moved from a best practice to a regulatory expectation in multiple sectors since a 2021 US Executive Order directed federal agencies to require SBOMs from software vendors. The EU Cyber Resilience Act (taking effect from 2027) includes explicit SBOM requirements. An SBOM is a machine-readable inventory of all software components — every dependency, its version, its license, and ideally its provenance (where it came from and how it was built).

SBOM Standard	Format	Primary Use Case
SPDX (Software Package Data Exchange)	RDF, JSON, YAML, tag-value text	License compliance tracking; US federal contracts
CycloneDX	XML, JSON, Protobuf	Security-focused; vulnerability management, attestation
SWID Tags	XML	Enterprise software inventory and patch management

SCA tools generate SBOMs in SPDX and CycloneDX formats. If your organization is a software vendor, you may be contractually required to provide SBOMs to customers and government agencies. If you are a software consumer — which every organization is — requiring SBOMs from your own vendors gives you visibility into the supply chain risk embedded in software you did not build.

11. Dependency Health Signals — Predicting Future Risk

AI-powered SCA tools go beyond known CVEs to assess dependency health signals that predict future vulnerability risk. A dependency that is not currently vulnerable but exhibits poor health signals is a future liability:

- **Maintenance activity:** When was the last commit? Is the project responding to issue reports? Long maintenance gaps predict unmaintained packages that accumulate unpatched vulnerabilities.
- **Contributor diversity:** A project with a single maintainer is a single point of failure — one compromised account, one hostile maintainer, one burned-out developer away from abandonment or compromise.
- **Release cadence:** Is the project keeping its own dependencies current? A project that has not updated its dependencies for two years is accumulating transitive vulnerability debt.
- **Community health:** Stars, forks, contributor growth, and issue response time are proxies for project vitality.
- **Adoption trajectory:** Is usage growing or declining? Declining adoption predicts reduced maintenance investment.

Proactive dependency hygiene — replacing unmaintained or unhealthy packages before they become vulnerable — prevents the firefighting that reactive-only SCA creates. The XZ Utils attacker specifically targeted a widely-depended-upon but thinly-maintained project, recognizing that a small contributor base with limited review capacity would be easier to infiltrate.

12. Private Package Registries — The Supply Chain Firewall

Pulling packages directly from public registries in CI builds — consuming from npm.com, pypi.org, or Maven Central at build time — exposes your pipeline to whatever is published in those registries at that

moment, including newly published malicious packages, typosquatting attacks, and packages with recently discovered vulnerabilities. Private registries (JFrog Artifactory, AWS CodeArtifact, GitHub Packages, Sonatype Nexus) serve as a supply chain firewall between your builds and public registries.

Approved packages are vetted before being mirrored into the private registry. New packages must pass review before your builds can consume them. Every package consumed in every build is auditable — you know exactly what was used in each build. Build reproducibility is improved: public registries can delete or modify packages; your private registry retains the exact versions used in each release. In regulated industries, private registries are often a compliance requirement for build reproducibility and supply chain integrity.

Key Terms Glossary

Term	Definition
SCA	Software Composition Analysis — identifying, inventorying, and assessing risk in open-source and third-party dependencies.
Transitive dependency	A dependency of a dependency — a package your code imports indirectly through another package's requirements.
Direct dependency	A package your code explicitly imports and lists in its package manifest (package.json, requirements.txt, pom.xml).
Reachability analysis	Call-graph analysis determining whether vulnerable functions in a dependency are actually invoked by your application code.
SBOM	Software Bill of Materials — a machine-readable inventory of all software components, versions, licenses, and provenance.
SPDX	Software Package Data Exchange — an SBOM standard format used for license compliance tracking and US federal requirements.
CycloneDX	Security-focused SBOM standard format used for vulnerability management and software attestation.
Dependabot	GitHub's automated dependency update tool — generates pull requests to update vulnerable dependencies at CVE disclosure time.
Continuous SCA monitoring	Real-time correlation of your dependency inventory against live vulnerability feeds, alerting immediately on new disclosures.
Typosquatting	Publishing malicious packages under names similar to popular packages to trick developers into importing them by mistake.
XZ Utils backdoor (2024)	Supply chain attack where a malicious maintainer inserted a backdoor into a widely-used Linux compression library after years of legitimate contribution.
Private package registry	An organization-controlled package mirror (Artifactory, CodeArtifact) that vets packages before they enter the build supply chain.
OSV	Open Source Vulnerabilities — a standardized vulnerability database aggregating security advisories across open-source ecosystems.
Copyleft license	Open-source license (GPL, AGPL) requiring derivative works to be released under compatible terms — creates obligations in commercial software.

Quick Revision Checklist

- Most modern application code is third-party dependency code — SCA makes the invisible dependency iceberg visible.
- XZ Utils (2024) is the exam-relevant SCA case study: attacker spent two years building trust before inserting a backdoor into a transitive dependency of hundreds of millions of systems.
- Reachability analysis reduces actionable SCA findings by 70-90% by determining whether vulnerable functions are actually called.
- Transitive dependencies represent the majority of dependency surface area and are largely invisible without SCA tooling.
- Point-in-time scanning vs. continuous monitoring: continuous monitoring is required to detect CVEs disclosed after the last scan.
- License compliance: GPL/AGPL require derivative works under compatible licenses; SSPL restricts cloud service use; MIT/Apache 2.0 are permissive.
- Automated PR generation (Dependabot, Renovate, Snyk) prevents backlog accumulation by proposing fixes at CVE disclosure time.
- SCA policy gates enforce dependency security governance automatically at the PR and deployment level.
- SBOM (SPDX or CycloneDX format) is now a regulatory expectation in the US (post-2021 Executive Order) and EU (Cyber Resilience Act).
- Private package registries (Artifactory, CodeArtifact) vet packages before they enter the build supply chain — a supply chain firewall.
- Dependency health signals (maintenance activity, contributor diversity, release cadence) predict future vulnerability risk.
- Know the key SCA tools: GitHub Dependabot, Snyk, Mend (formerly WhiteSource), Endor Labs, Socket, OWASP Dependency-Check.

10 Exam-Based MCQs — Software Composition Analysis (SCA)

Test yourself after reading. These questions are written in real exam style — scenario-heavy, with plausible distractors. Read every explanation, including for questions you answered correctly.

Q1. *Scenario:* A security team is reviewing the XZ Utils supply chain attack from 2024. The attacker spent two years making legitimate contributions to the project before inserting a backdoor. No CVE existed during the attack period — the compromise was discovered by chance through performance anomalies, not through vulnerability scanning. The team is evaluating which SCA capabilities would have provided earlier detection. Which SCA capability would MOST directly have provided early warning for this type of attack?

- A) Point-in-time vulnerability scanning against the NVD at the last CI build
- B) AI-powered monitoring for unusual maintainer activity patterns and unexpected changes in established packages' build scripts
- C) License compliance scanning to identify GPL-licensed dependencies in the production stack
- D) Automated pull request generation to update all dependencies to their latest versions weekly

Answer: B **Explanation:** B is correct because the XZ Utils attack was characterized by a new contributor rapidly gaining commit access (anomalous maintainer activity) and inserting a backdoor into the build system rather than the source code (unexpected change in build scripts of an established package). AI monitoring of these behavioral signals — which are distinct from CVE databases — is the control best suited to detecting this attack pattern before a CVE is published. A would not have detected it because no CVE existed during the attack period. C addresses license compliance, not supply chain compromise. D addresses keeping packages current but would not detect a compromised maintainer.

Q2. An SCA scan of a Java application returns 847 vulnerability findings across all dependencies. The security team determines only 23 of these represent actual risk. What SCA capability would MOST effectively reduce this noise in future scans?

- A) Upgrading to a newer version of the SCA tool
- B) Excluding all transitive dependencies from future scans to focus only on direct imports
- C) Enabling reachability analysis to filter findings to only vulnerabilities in code paths actually invoked by the application
- D) Accepting all findings as false positives and resetting the baseline to zero

Answer: C **Explanation:** C is correct because reachability analysis performs call-graph analysis to determine whether your application code actually invokes the vulnerable functions in each dependency — a critical CVE in a library function your code never calls is not an exploitable risk. Reachability analysis typically reduces actionable findings by 70-90%, bringing 847 findings to a more manageable actionable set. A improves tool capability generally but does not specifically address the noise problem. B excludes transitive dependencies entirely — removing a major portion of your actual attack surface, which is the opposite of responsible SCA practice. D does not reduce noise — it creates false assurance.

Q3. A startup's legal counsel flags that one of the Python packages used in their commercial SaaS product is licensed under AGPL-3.0. What is the MOST significant legal concern?

- A) AGPL-3.0 requires the company to pay royalties to the package maintainer for commercial use
- B) AGPL-3.0 requires that the complete source code of the SaaS product, including proprietary components, be made available to users under compatible terms
- C) AGPL-3.0 prohibits use of the package in production environments without written permission from the maintainer
- D) AGPL-3.0 requires the company to contribute all bug fixes back to the package within 30 days

Answer: B **Explanation:** B is correct because AGPL-3.0 (Affero GPL) specifically extends the GPL copyleft requirement to network-delivered software: if you offer a service using AGPL-licensed software, users who interact with that service are entitled to receive the complete source code, including your proprietary components, under AGPL-compatible terms. This is the specific risk for SaaS products that GPL alone did not clearly address. A is incorrect — AGPL does not require royalty payments; it is an open-source license. C is incorrect — AGPL does not prohibit commercial or production use; it requires source disclosure. D is incorrect — the contribution-back requirement is not a 30-day provision; the obligation is source disclosure upon request, not proactive contribution.

Q4. What is the PRIMARY difference between point-in-time SCA scanning and continuous SCA monitoring?

- A) Point-in-time scanning covers all dependencies; continuous monitoring only covers direct dependencies
- B) Continuous monitoring correlates your dependency inventory against live vulnerability feeds and alerts immediately when new CVEs affect your packages, while point-in-time scanning only reflects the vulnerability state at the moment of the scan
- C) Point-in-time scanning is performed by developers; continuous monitoring requires a dedicated security operations team
- D) Continuous monitoring is only available for JavaScript and Python ecosystems; point-in-time scanning covers all languages

Answer: B **Explanation:** B is correct because the key operational difference is temporal coverage: point-in-time scanning tells you about vulnerabilities that existed when the scan ran; continuous monitoring alerts you the moment a new vulnerability is disclosed that affects packages in your current inventory — potentially the same day the CVE is published. A is incorrect — both approaches can cover direct and transitive dependencies; this is not the distinguishing characteristic. C is incorrect — tool operation model is not the fundamental distinction. D is incorrect — both approaches are available across multiple language ecosystems.

Q5. Which statement BEST describes a Software Bill of Materials (SBOM)?

- A) A financial document listing the licensing costs of all commercial software used in an application
- B) A machine-readable inventory of all software components in a product, including dependency versions, licenses, and provenance
- C) A government regulatory document that software vendors must submit to obtain export approval
- D) A test report documenting that all software components have passed security vulnerability assessments

Answer: B **Explanation:** B is correct because an SBOM is specifically a machine-readable inventory of software components — the what, where, and provenance of every package in a software product. Standard formats include SPDX and CycloneDX. A describes a software licensing cost document, which is unrelated to SBOM. C describes an export control filing, unrelated to SBOM. D describes a security assessment report, not an inventory document.

Q6. A Node.js application imports the 'express' framework directly. The 'express' framework internally imports 'qs' for query string parsing. A critical CVE is disclosed in 'qs'. How is 'qs' classified in this application's dependency structure, and what is the implication?

- A) Direct dependency — the application must explicitly update its package.json to reference qs
- B) Transitive dependency — the vulnerability must be addressed by updating express (which pulls the patched qs version) or by explicitly overriding the qs version

- C) Orphaned dependency — qs can be removed from the node_modules directory without affecting the application
- D) Development dependency — the vulnerability only affects development environments and not production

Answer: B **Explanation:** B is correct because 'qs' is a transitive dependency — your application imports express, express imports qs, but your application has no direct dependency declaration for qs. To remediate, you either update express to a version that depends on the patched qs, or explicitly add qs as a direct dependency at the patched version (an override or resolution in npm terms). A is incorrect — qs is not listed in the application's package.json, so it is not a direct dependency. C is incorrect — qs is an active transitive dependency used by express and cannot be removed without breaking express functionality. D is incorrect — nothing in the scenario indicates qs is a development dependency; the classification is transitive, not dev.

Q7. An organization wants to prevent typosquatting attacks where developers accidentally import malicious packages with names similar to legitimate libraries. Which control BEST addresses this?

- A) Training developers to verify package spellings before adding them to projects
- B) Implementing a private package registry that only allows vetted, approved packages to be consumed in builds
- C) Enabling two-factor authentication on all developer npm accounts
- D) Running point-in-time SCA scans weekly to detect newly introduced malicious packages

Answer: B **Explanation:** B is correct because a private package registry (Artifactory, CodeArtifact) acts as a supply chain firewall — builds can only consume packages that have been pre-approved through a vetting process. Typosquatted packages that were never approved cannot enter the build pipeline regardless of developer mistakes. A relies on human vigilance, which is inherently unreliable at scale. C protects developer npm account credentials but does not prevent accidental imports of malicious packages that are already in the public registry. D detects after the fact but does not prevent initial import of malicious packages.

Q8. GitHub Dependabot is configured for a production Python application. Which sequence of events BEST describes how Dependabot reduces mean time to remediate dependency vulnerabilities?

- A) Dependabot notifies the CISO by email → CISO assigns a ticket → developer researches the vulnerability → developer implements the fix
- B) Dependabot detects a new CVE affecting a dependency → automatically generates a pull request updating the dependency to the patched version → runs automated tests → developer reviews and merges
- C) Dependabot blocks all deployments until every vulnerable dependency is addressed → developer manually updates all dependencies → Dependabot re-enables deployment
- D) Dependabot generates a weekly digest of all dependency vulnerabilities → security team triages manually → developer implements fixes in next sprint

Answer: B **Explanation:** B is correct because Dependabot's automated PR workflow reduces developer

effort to reviewing a proposed fix rather than researching a vulnerability and determining the correct remedy — the highest-friction step in traditional remediation. Automated test execution within the PR shows whether the upgrade is safe. A describes a notification-only flow with no automation; it retains all the manual steps that create backlog. C is inaccurate — Dependabot generates PRs, not deployment blocks (policy gates in the CI/CD pipeline provide the blocking capability). D describes a periodic batch workflow that retains manual triage and sprint scheduling delays.

Q9. Which dependency health signal is MOST predictive of future supply chain attack risk, based on the XZ Utils backdoor pattern?

- A) A package has not released a new version in six months
- B) A widely-used package maintained by a very small contributor base accepts new maintainers with minimal vetting
- C) A package's documentation was last updated two years ago
- D) A package's GitHub star count has plateaued over the past year

Answer: B **Explanation:** B is correct because the XZ Utils attack specifically exploited a thinly-maintained project that accepted a new contributor with minimal vetting. Small contributor bases with informal onboarding processes are high-value targets for supply chain infiltration because gaining commit access requires building trust with only one or two people. A: A 6-month release gap may indicate maintenance slowdown but does not directly signal compromise risk. C: Outdated documentation is a quality signal but not a security compromise predictor. D: Stable star count is a community metric, not a supply chain security signal.

Q10. A software vendor is about to sign a contract with a US federal agency. The procurement officer requires an SBOM in a standard machine-readable format. Which formats should the vendor prepare, and what regulatory mandate underlies this requirement?

- A) CSV and Excel formats; required by FISMA for all federal software procurements since 2018
- B) SPDX or CycloneDX formats; driven by the 2021 US Executive Order on Improving the Nation's Cybersecurity (EO 14028)
- C) PDF and Word document formats; required by NIST SP 800-53 for all Category 1 software systems
- D) JSON and XML formats of the vendor's choice; required by the EU Cyber Resilience Act for all software sold in the US

Answer: B **Explanation:** B is correct because the 2021 US Executive Order 14028 (Improving the Nation's Cybersecurity) directed NIST and federal agencies to define and require SBOMs from software vendors, with SPDX and CycloneDX as the recognized standard machine-readable formats. A is incorrect — FISMA does not include a 2018 SBOM requirement; the SBOM regulatory push began with EO 14028 in 2021. C is incorrect — PDF and Word are not machine-readable SBOM formats; NIST SP 800-53 does not mandate SBOM format. D is incorrect — the EU Cyber Resilience Act applies to software sold in the EU, not the US; and the format requirement is SPDX/CycloneDX, not freely chosen JSON/XML.